Chapter 15

BOOLEAN SET OPERATIONS

An algorithm for evaluating Boolean set operations of solid objects is one of the most powerful facilities available in a solid modeler. In modelers based on boundary representations, the Boolean set operations algorithm is also the technically most demanding component. This chapter describes a Boolean set operations algorithm that can work for essentially all possible arrangements of polyhedral objects. As we shall see, the algorithm is similar in general structure to the splitting algorithm of the previous chapter.

15.1 INTRODUCTION

A Boolean set operations algorithm that can be used to unite, intersect, or subtract solid objects with each other is an essential component of any solid modeling system. For human users, set operations offer a tool for describing complex objects in terms of a series of operations on simpler components. For various algorithms, set operations offer a tool for describing physical processes applied to objects. For instance, an algorithm for verifying code for numerically controlled (NC) machine tools can use set operations to model effects in individual machining operations, or for determining collisions between the tool and the fixtures [58].

Unfortunately, Boolean set operations algorithms for boundary representations are in general plagued by two kinds of problems: First, to be effective, a set operations algorithm must be able to treat all possible kinds of geometric intersections that may appear between faces, edges, and vertices of the two objects. The proper treatment of all cases easily leads to a

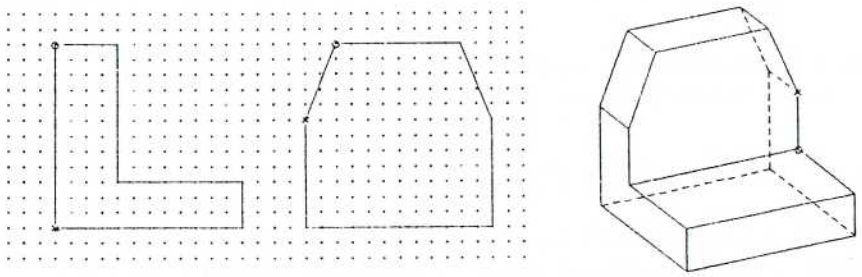

Figure 15.1 Set operation of two extruded profiles.

very hairy case analysis. Second, the very case analysis must be based on various tests for overlap, coplanarity, and intersection which are difficult to implement robustly in the presence of numerical errors.

To appreciate the difficulty of the set operations, consider Figure 15.1 that depicts the set intersection of two extruded profiles. In the figure, the two top views depict two orthogonal profiles. The solid shown in the bottom right view is generated by sweeping the profiles appropriately to generate overlapping solids, and calculating their Boolean intersection. Clearly, the swept objects intersect each other in various "special" ways. In this case a set operations algorithm will have difficulties if it is not prepared to deal with coplanar overlapping faces such as the bottom face of the resulting object.

Observe that the code for overlapping coplanar faces must be able to handle robustly a two-dimensional set operation of two polygons, a task that even alone is quite challenging for a programmer uninitiated in geometry. All the program codes required to break the problem into solvable cases, and the codes for each case sum up to a voluminous and complicated algorithm whose correctness is difficult to establish.

For the purposes of GWB, we shall seek a different path to complete set operations. The algorithm to be presented breaks the Boolean set operations problem into a set of simpler problems of just two major kinds that can be handled with a relatively straightforward case analysis. As to be elaborated in the subsequent sections, the main underlying tasks are very similar in nature with the vertex neighborhood classification of the splitting algorithm described in the previous chapter.

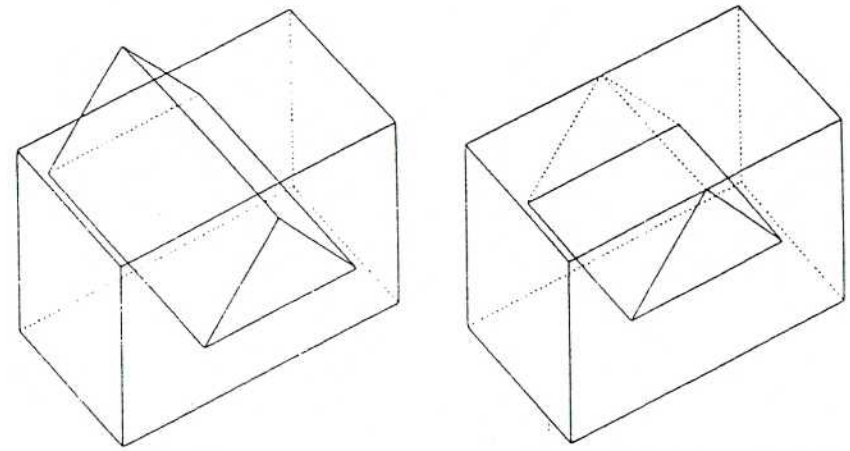

Figure 15.2 Set operations are not closed for 2-manifolds.

15.2 STATEMENT OF THE PROBLEM

The input of the Boolean set operations algorithm will consist of two half-edge data structures, A and B. Both A and B are assumed to be "correct" models of a three-dimensional solid object. In particular, they should be 2-manifolds, i.e., satisfy the following criteria:

1. Every edge belongs to exactly two flat faces.
2. Every vertex is surrounded by a single cycle of edges and faces.
3. Faces many not intersect each other except at common edges or vertices.

Our aim is to devise an algorithm that can calculate a new solid that represents the desired regularized Boolean combination of A and B, i.e., $A \cup^* B$, $A \cap^* B$, or $A \setminus^* B$, where $\cup^*$, $\cap^*$, and $\setminus^*$ are the regularized counterparts of the ordinary (point) set operations union $\cup$, intersection $\cap$, and set difference $\setminus$.

Unfortunately, we cannot require that the resulting solid will always satisfy both of the requirements 1 and 3 above. This is so because 2-manifolds are not closed under set operations, i.e., a Boolean combination of two 2-manifolds is not necessarily a 2-manifold. For instance, the set difference of the block and the wedge of Figure 15.2 has four faces meeting at an edge, and criterion 1 is not satisfied.

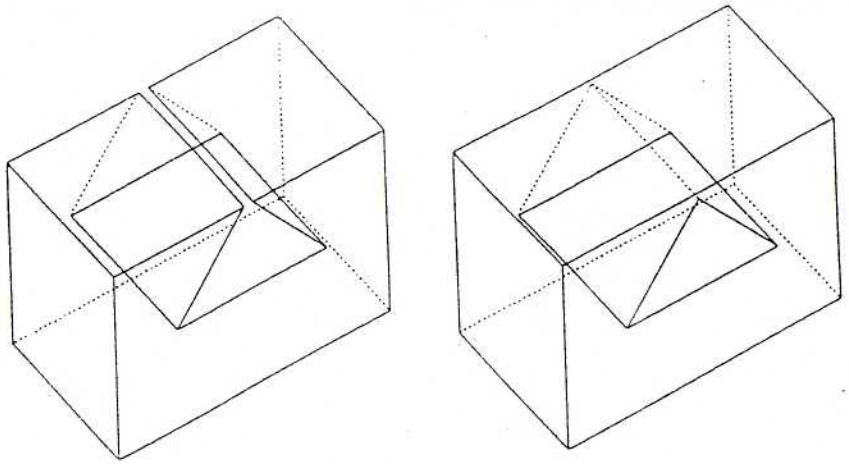

Figure 15.3 Pseudomanifold representations of a nonmanifold.

We accept this limitation because a winged-edge data structure cannot represent objects such as the result of Figure 15.2 directly. The only resort available is to represent the nonmanifold as a "pseudomanifold," i.e., as if it was one or the other of the objects depicted in Figure 15.3. In particular, in the case on the left the wedge extends outside the top face of the block, while on the right the wedge only intersects the front and the back faces of the block. In the first case, the object has two entirely coincident edges, while in the second case it has an edge lying on a face.

Hence, we shall require that the result of the set operations algorithm satisfies all other criteria except 3. Instead of 3, a more permissive criterion will be required:

3′. Faces of the polyhedron may not intersect each other at their internal points. The only other kinds of intersections are the following:

- Some edges of the polyhedron are entirely coincident. In this case, the edge neighborhoods of these edges must be distinct.
- Some edges may lie on a face. In this case, both faces adjacent to the edge must lie on the same side of the face.
- Some vertices may lie on a face. In this case, all edges and faces adjacent to the vertex must lie on the same side of the face.
- Some vertices may lie on an edge or on a vertex. In these cases, all edges and faces adjacent to the vertex must lie on the same side of the surface defined by the faces of the incident edge or vertex.

That is, the surface of the resulting object may "touch" itself at some edges or vertices, but not intersect itself properly.

15.3 BOUNDARY CLASSIFICATION

The splitting algorithm of Chapter 14 is based on reducing the global problem into a collection of local vertex neighborhood classification problems, and processing each vertex neighborhood according to rules that guarantee the correctness of the result. The set operations algorithm and its classifier will follow the same general approach.

To see the close relationship between the splitting problem and the set operations, recall that the splitting algorithm divides a solid S in two sets ($Above$, $Below$) according to a splitting plane SP. For set operations we shall need the more general splitting of a polyhedron A according to a reference polyhedron B. More rigorously, let us denote the bounding surface of A by $b(A)$. Then the two objects resulting from the general splitting operation will be denoted by $AinB$ (for the part of $b(A)$ inside B) and $AoutB$ (for the part of $b(A)$ outside B).[1]

Given two solids A and B, the collection of four objects $AinB$, $AoutB$, $BinA$, and $BoutA$ formed by splitting A against B and symmetrically B against A are here called the *boundary classification* of A and B. For instance, Figure 15.4(a) depicts two intersecting bricks; their intersection curve is indicated with heavy lines. The resulting boundary classification is shown in (b). (Observe that each component of the classification can be represented as a solid in its own right that generally has some nonplanar "intersection" faces.)

From the boundary classification of A and B, all their Boolean combinations are readily computed:

$$\begin{aligned} A \cup B &= AoutB \oplus BoutA \\ A \cap B &= AinB \oplus BinA \\ A \setminus B &= AoutB \oplus (BinA)^{-1} \end{aligned} \tag{15.1}$$

where $\oplus$ denotes the gluing operation of Chapter 12, and $(BinA)^{-1}$ denotes the "complement" of $BinA$, i.e., $BinA$ with the orientation of all faces reversed.

[1] Note that this is exactly what the splitting algorithm does if we think of the halfspace "above" SP as a polyhedron B bounded by just one infinite face.

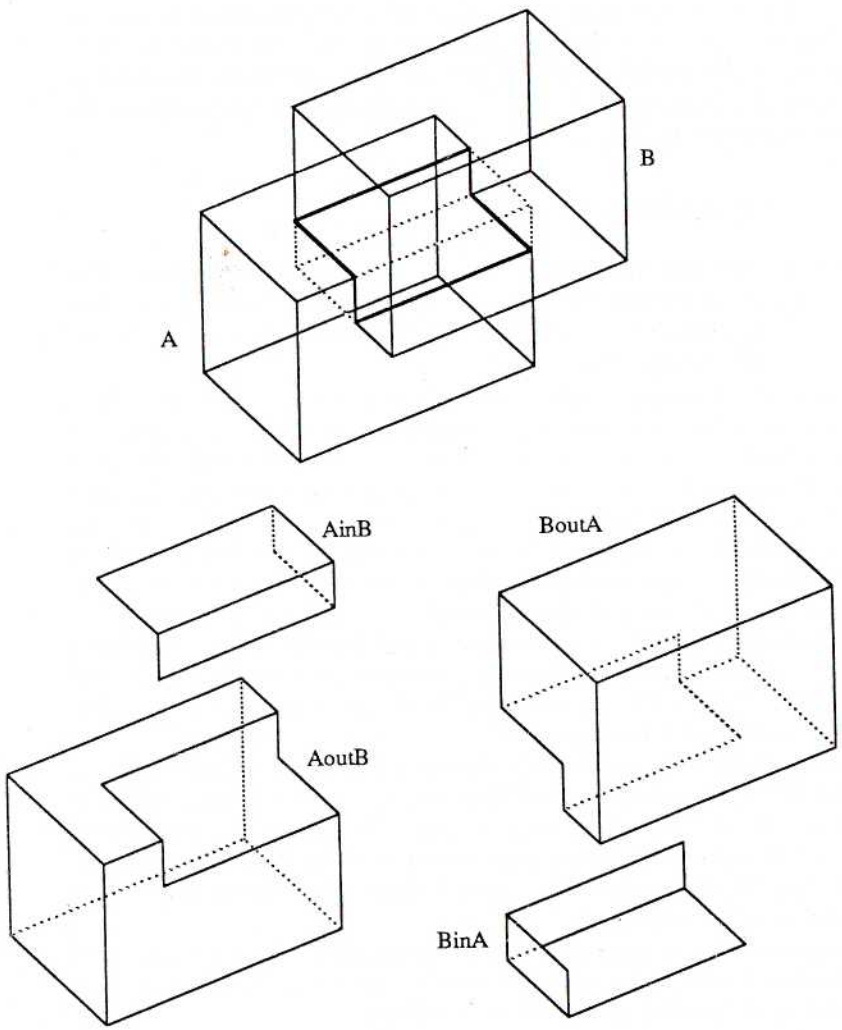

Figure 15.4 Boundary classification of two bricks.

In the splitting algorithm, the vertex neighborhood classifier treats edges and faces lying on the splitting plane by reclassifying them as lying above or below the plane. The reclassification rules are designed so as to guarantee the regularity of the result. A similar approach can be followed also in the classifier for set operations. However, in this case the reclassification rules will be somewhat more complicated.

To provide insight into the reclassification rules, it is useful to consider the *8-way boundary classification* of A and B that adds the components $AonB^+$, $AonB^-$, $BonA^+$, and $BonA^-$ to the components of the 4-way classification ($AinB$, $AoutB$, $BinA$, $BoutA$). The component $AonB^+$ consists of those parts of $b(A)$ that lie on $b(B)$ so that the face normals of the respective faces are equal, while $AonB^-$ consists of the overlapping parts where the normals are opposite. The components $BonA^+$ and $BonA^-$ are defined analogously.

Figure 15.5 illustrates the 8-way boundary classification of two objects A and B shown in three views. For clarity, the "in"- and "on"-components of the classification are shaded.

From the components of the 8-way classification, the result of a set operation can be computed as follows:

$$\begin{aligned} A \cup B &= AoutB \otimes BoutA \otimes AonB^+ \\ A \cap B &= AinB \otimes BinA \otimes AonB^+ \\ A \setminus B &= AoutB \otimes (BinA)^{-1} \otimes AonB^- \end{aligned} \tag{15.2}$$

(The reader is encouraged to check that these equations indeed give the correct regularized results for the objects of Figure 15.5.)

It would be perfectly possible to construct a set operations algorithm that computes the full 8-way boundary classification, and works out the result according to Equations 15.2. Because of the inherent burden of this computation, we do not follow this approach directly. Instead, our algorithm calculates a special 4-way boundary classification where the "on"-components of the 8-way classification are lumped with the components of the 4-way classification so as to make Equations 15.1 and 15.2 equivalent.

A set of reclassification rules with this property can be written as follows:

Op	$AonB^+$	$AonB^-$	$BonA^+$	$BonA^-$
$\cup$	$AoutB$	$AinB$	$BinA$	$BinA$
$\cap$	$AinB$	$AoutB$	$BoutA$	$BoutA$
$\setminus$	$AinB$	$AoutB$	$BoutA$	$BoutA$

(15.3)

For instance, the first row indicates that for set union, $AonB^+$ should be treated as being a part of $AoutB$, and $AonB^-$ as a part of $AinB$. $BonA^+$

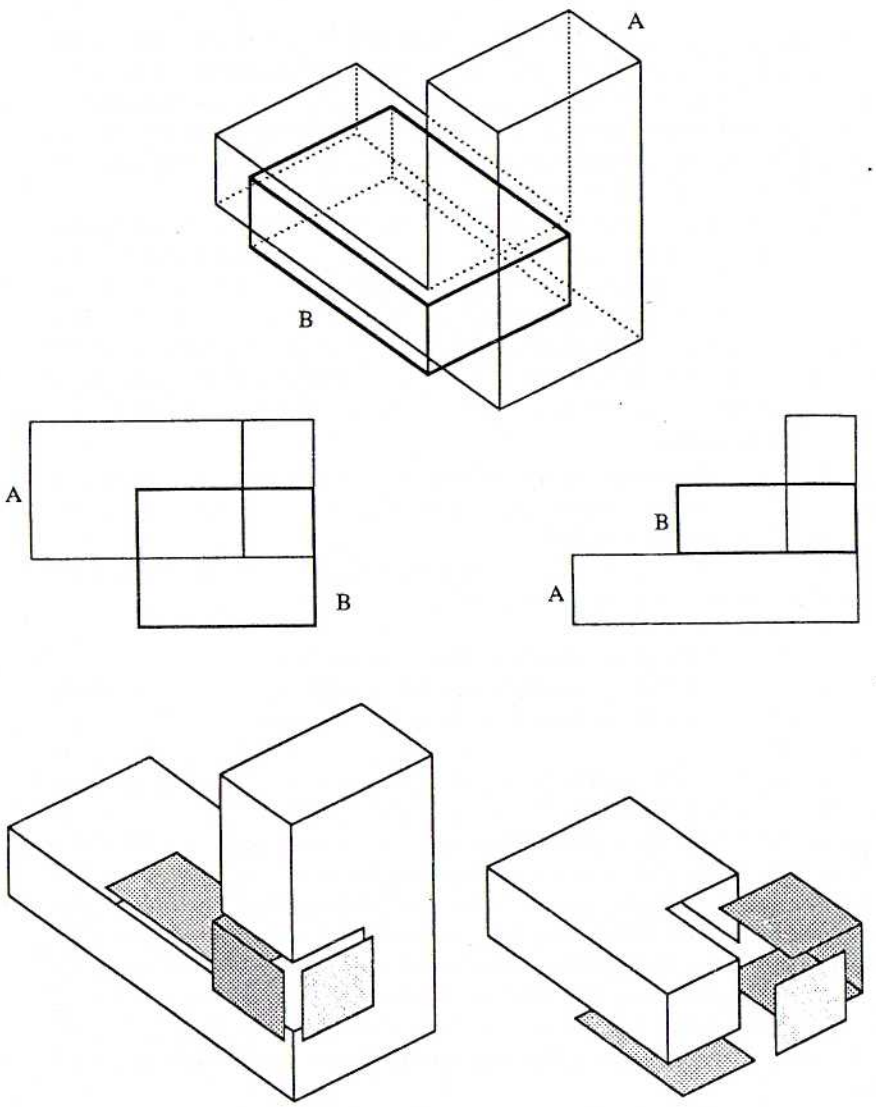

Figure 15.5 8-way boundary classification of two polyhedra.

and $BonA^-$ are both treated as a part of $BinA$. Hence, only $AonB^+$ will appear in the final result.

It should be stressed that these rules will be implemented directly in the vertex neighborhood classifier routines, and that the higher levels of the algorithm can effectively ignore all "on"-cases. (Observe the analogy of the rules of Equations 15.3 and the reclassification of "on"-edges in the splitting algorithm of Chapter 14.)

15.4 THE ALGORITHM

Let us now start the description of the set operations algorithm. The implementation of Program 15.1 is written in a style stressing the close relationship of splitting and set operations; the reader is urged to compare it with the splitting algorithm of Program 14.1.

For use of the vertex neighborhood classifier, the algorithm uses three arrays *sonvv*, *sonva*, and *sonvb* for representing the set of conincident vertices of A and B, the set of vertices of A on a face of B, and the set of vertices of B on a face of A, respectively. Similarly to the splitting algorithm, the set operations algorithm will also need arrays for representing the sets of null edges (*sonea*, *soneb*) to be generated on A and B, and the sets of null faces (*sonfa*, *sonfb*). All these arrays are assumed to be global data available to all parts of the algorithm.

The algorithm takes four arguments. A and B point at the solids to be combined by means of the set operation *op* represented by means of literal values `UNION`, `INTERSECT`, and `DIFFERENCE`. The argument *res* will be filled with a pointer to the resulting solid.

The algorithm first evaluates all face equations and arranges the global variables *maxv* and *maxf* so that they can be used to generate globally unique new identifiers for vertices and faces. The procedure **updmaxnames** is assumed to increment the face and vertex identifiers of its argument solid so that they do not overlap the ranges 1–*maxf* and 1–*maxv*, and update *maxv* and *maxf* appropriately.

The procedures **setopgenerate**, **setopclassify**, **setopconnect**, and **setopfinish** have similar roles as their counterparts in Program 14.1. They will be elaborated in the following sections.

15.5 REDUCTION STEP

First, let us consider the procedure **setopgenerate** responsible for reducing the set operations problem into a collection of vertex neighborhood classification problems. This task can be formulated as follows:

```
struct
{
        Vertex  *va, *vb;
}       sonvv[MAXONVERTICES];
int     nvtx;
struct
{
        Vertex *v;
        Face   *f;
}       sonva[MAXONVERTICES], sonvb[MAXONVERTICES];
int     nvtxa, nvtxb;
Edge    *sonea[MAXNULLEDGES], *soneb[MAXNULLEDGES];
int     nedga, nedgb;
Face    *sonfa[MAXNULLFACES*2], *sonfb[MAXNULLFACES*2];
int     nfaca, nfacb;

int     setop(A, B, res, op)
Solid   *A, *B, **res;
int     op;
{
        Face    *f;

        for(f = A->sfaces; f != NIL; f = f->nextf)
                faceeq(f->flout, f->feq);
        for(f = B->sfaces; f != NIL; f = f->nextf)
                faceeq(f->flout, f->feq);
        getmaxnames(A);
        updmaxnames(B);
        setopgenerate(A, B);
        setopclassify(op);
        if(nedga == 0)
        {
                printf("setop: no intersections found\n");
                return(0);
        }
        setopconnect();
        setopfinish(A, B, res, op);
        return(1);
}
```

Program 15.1 Outline of the set operations algorithm.

1. Locate all pairs of edges e_A of A and e_B of B that intersect each other properly, i.e., at an internal point of both edges. Subdivide both edges at their intersection point, i.e., replace each edge by two edges and a new vertex lying at the intersection point.

2. Locate all edges of A that pass through a vertex of B. Subdivide all such edges at the intersection point.

3. Do step 2 symmetrically for edges of B and vertices of A.

4. Locate all coincident pairs of vertices v_A of A and v_B of B, and store them for later processing. (Of course, the resulting set includes at least all vertices added during steps 1 through 3.)

5. Locate all edges e_A of A that intersect a face f_B of B properly, i.e., at an internal point of f_B. Subdivide all such edges at the intersection point.

6. Do step 5 symmetrically for edges of B and faces of A.

7. Locate all vertices v_A of A that lie within a face f_B of B and store the pair (v_A, f_B) for later processing. (This set will include all vertices added during step 5.)

8. Locate all vertices v_B of B that lie within a face f_A of A and store the pair (v_B, f_A) for later processing. (This set includes all vertices added in step 6.)

The reader is urged to compare these steps with those of the splitting algorithm of Chapter 14. In particular, note the analogy of steps 5–8 of this algorithm and the splitting algorithm.

To implement the steps 1–8 as a real procedure it is very helpful to make the further assumption that all faces of A and B are "maximal," i.e., that all coplanar neighbor faces have been combined, and all "inessential" edges that occur within a single face have been removed. Under this assumption, the search of intersecting entities can be organized as a comparison between all edges of one solid against the faces of the other solid and vice versa, and applying the following case analysis for each edge e and face f:

1. The vertices of e are on the same side of the plane of f. In this case, e does not intersect f.

2. The vertices of e are on different sides of the plane of f. That is, e intersects the plane "properly," In this case, an intersection point can be calculated and classified against f by means of the vertex-in-face procedure `contfv` of Chapter 13. This gives a second-level case

```
void      setopgenerate(A, B)
Solid     *A, *B;
{
     Edge      *e;

     nvtx = nvtxa = nvtxb = 0;
     for(e = A->sedges; e != NIL; e = e->nexte)
          processedge(e, B, 0);
     for(e = B->sedges; e != NIL; e = e->nexte)
          processedge(e, A, 1);
}

void      processedge(e, s, BvsA)
Edge      *e;
Solid     *s;
int       BvsA;
{
     Face      *f;

     for(f = s->sfaces; f != NIL; f = f->nextf)
          dosetopgenerate(e, f, BvsA);
}
```

Program 15.2 Generation of relevant vertices.

analysis according to whether the intersection lies within the face, at an edge of the face, at a vertex, or without the face.

3. One or both of the vertices of e lie on the plane of f. Similar to the case 2 above, they can be classified with **contfv**.

Except for the vertices, the case of e being coplanar with f can be ignored: all relevant intersections between e and the edges of f will be located when comparing e against the (noncoplanar) neighbor faces of f.

The edge-face comparison is realized in Program 15.2 ultimately by calling the procedure **dosetopgenerate** of Program 15.3 that implements the above case analysis. The procedure first computes the signed distances of the endpoints of the query edge e from the plane of the face f. If the signs of the distances differ, the edge must intersect the plane, and the intersection point $p = (x\ y\ z)$ is computed.

This leads us to the second level of the case analysis. First, the inclusion of p within f is examined by means of the procedure **contfp**, a variation


```
extern HalfEdge *hithe;
extern Vertex *hitvertex;

void      dosetopgenerate(e, f, BvsA)
Edge      *e;
Face      *f;
int       BvsA;
{
     Vertex        *v1, *v2;
     double        d1, d2, t, x, y, z;
     int           s1, s2, cont;

     v1 = e->he1->vtx;
     v2 = e->he2->vtx;
     s1 = comp((d1 = dist(v1->vcoord, f->feq)), 0.0, EPS);
     s2 = comp((d2 = dist(v2->vcoord, f->feq)), 0.0, EPS);
     if(s1 == -1 && s2 == 1 || s1 == 1 && s2 == -1)
     {
          t = d1 / (d1 - d2);
          x = v1->vcoord[0] + t * (v2->vcoord[0] - v1->vcoord[0]);
          y = v1->vcoord[1] + t * (v2->vcoord[1] - v1->vcoord[1]);
          z = v1->vcoord[2] + t * (v2->vcoord[2] - v1->vcoord[2]);
          if((cont = contfp(f, x, y, z)) > 0)
          {
               lmev(e->he1, e->he2->nxt, ++maxv, x, y, z);
               if(cont == 1)
                    addsovf(e->he1->vtx, f, BvsA);
               else if(cont == 2)
               {
                    lmev(hithe, mate(hithe)->nxt, ++maxv, x, y, z);
                    addsovv(e->he1->vtx, hithe->vtx, BvsA);
               }
               else if(cont == 3)
                    addsovv(e->he1->vtx, hitvertex, BvsA);
               processedge(e->he1->prv->edg, f->fsolid, BvsA);
          }
     }
     else
     {
          if(s1 == 0) dovertexonface(v1, f, BvsA);
          if(s2 == 0) dovertexonface(v2, f, BvsA);
     }
}
```

Program 15.3 Edge-face comparison.

```
void      dovertexonface(v, f, BvsA)
Vertex    *v;
Face      *f;
int       BvsA;
{
     int          cont;

     if((cont = contfv(f, v)) == 1)
          addsovf(v, f, BvsA);
     else if(cont == 2)
     {
          lmev(hithe, mate(hithe)->nxt, ++maxv,
               v->vcoord[0], v->vcoord[1], v->vcoord[2]);
          addsovv(v, hithe->vtx, BvsA);
     }
     else if(cont == 3)
          addsovv(v, hitvertex, BvsA);
}
```

Program 15.4 Vertex-face comparison.

of the procedure contfv that tests for the inclusion of a point $(x\ y\ z)$ in a face f. If p intersects f is some fashion (i.e., contfp > 0), e is subdivided at p by creating a new vertex v on it. If contfp $= 1$, p is known to lie properly within f, and the pair v, f is inserted in the proper set of coplanar vertex-face pairs by means of the procedure **addsovf**.

If contfp $= 2$, p lies on an edge e' of f available in the external variable *hithe* (see Program 13.4, page 223). In this case, also e' is subdivided, and the pair of new vertices is inserted to the set of coincident vertices by means of the procedure addsovv. Finally, if p lies at a vertex v' of f (available in the external variable *hitvertex*), the pair v, v' is inserted.

If one or other of the distances of the endpoints of the query edge is (approximately) zero, a coplanar vertex has been located. They are handled by the procedure dovertexonf of Program 15.4 which is similar in structure to the Program 15.3, and is presented without further explanations.

Observe that after subdividing the query edge, **dosetopgenerate** will recursively call itself through the procedure **processedge** of Program 15.3 to make sure that all intersections of the subdivided edge with the solid are processed. Note also that the procedures **addsovf** and **addsovv** should check against duplicate inputs, because the objects to be stored will be encountered several times.


```
void    setopclassify(op)
int     op;
{
        int     i;

        nedga = nedgb = 0;
        for(i=0; i<nvtxa; i++)
                vtxfacclassify(sonva[i].v, sonva[i].f, op, 0);
        for(i=0; i<nvtxb; i++)
                vtxfacclassify(sonvb[i].v, sonvb[i].f, op, 1);
        for(i=0; i<nvtx; i++)
                vtxvtxclassify(sonvv[i].va, sonvv[i].vb, op);
}
```

Program 15.5 Vertex neighborhood classification.

15.6 VERTEX NEIGHBORHOOD CLASSIFIER

After the reduction step, the set operations problem is reduced into a collection of vertex neighborhood classification problems of two kinds:

1. *Vertex-face classification*: The processing of a vertex of one solid that lies on a face of the other solid.

2. *Vertex-vertex classification*: The processing of a pair of coincident vertices of A and B.

In strict analogy to the splitting algorithm, both of these problems are solved by a neighborhood classifier that inserts the proper null edges so as to split the vertex neighborhoods into subcycles inside and outside of the respective other polyhedron.

15.6.1 Vertex-Face Classification

In Program 15.5, the procedure vtxfacclassify serves the role of the vertex-face classifier. This procedure is essentially identical to the corresponding procedure for the splitting algorithm (Programs 14.3–7). The differences are the following:

1. Instead of the splitting plane SP, the plane of the face is used, and the notions of "above" and "below" are replaced by "in" and "out."

2. "On"-sectors are reclassified according to Equations 15.3 instead of the rules of Chapter 14.

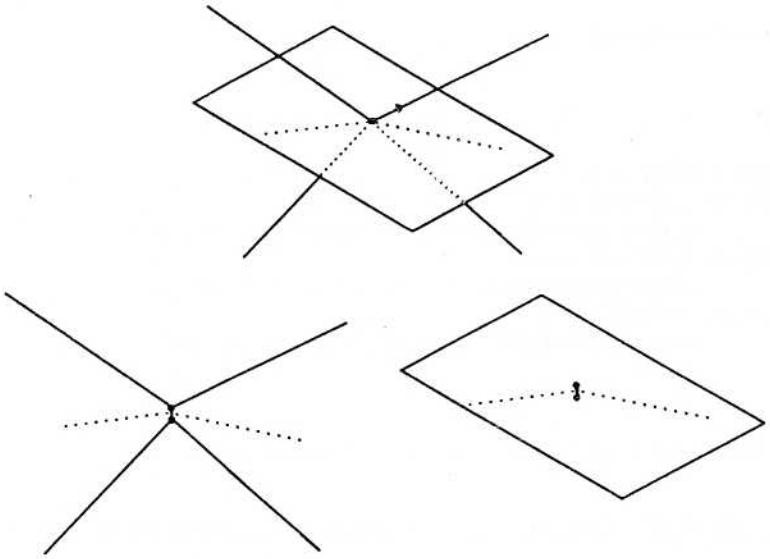

Figure 15.6 Vertex-face classification.

3. In addition to splitting the vertex neighborhood, a reference to the intersection point must also be stored within the face so that the intersection polygon(s) can be drawn through the point.

As for the last item, the algorithm for joining null edges can work properly if the intersections are marked by inserting null edges as an internal loop into the face intersected; see Figure 15.6.

For space reasons, we shall not elaborate the vertex-face classifier any further.

15.6.2 Vertex-Vertex Classification

The vertex-vertex classifier is expected to compare two vertex neighborhoods (with coincident base vertices), determine their intersection, and insert the appropriate null edges so that the intersection polygons can be correctly generated. As this component forms the core of the algorithm, we shall present it in considerable detail.

An outline of the vertex-vertex classifier is shown in Program 15.6; the reader is urged to compare it with the analogous Program 14.3. The


```
# define OUT -1
# define IN 1
# define ON 0

struct nb
{
        HalfEdge        *he;
        vector          ref1;
        vector          ref2;
        vector          ref12;
} nba[MAXSECTORS], nbb[MAXSECTORS];
int     nnba, nnbb;

struct
{
        int             secta, sectb;
        int             s1a, s2a;
        int             s1b, s2b;
        int             intersect;
}       sectors[MAXSECTORS];
int     nsectors;

void    vtxvtxclassify(va, vb, op)
Vertex  *va, *vb;
int     op;
{
        setopgetneighborhood(va, vb);
        sreclsectors(op);
        srecledges(op);
        sinsertnulledges();
}
```

Program 15.6 Vertex-vertex neighborhood classifier.

algorithm uses three arrays *nba*, *nbb*, and *sectors* to represent the sectors of the neighborhood of the two vertices and the intersections between sectors; these arrays are assumed to be accessible to all algorithms of this section.

Search for intersecting sectors

Recall how the classifier for the splitting problem could work by locating all sectors of a neighborhood that intersect the splitting plane, and dividing the neighborhood along the intersecting sectors. Analogously, the vertex-vertex classifier of Program 15.7 works by considering pairs of sectors from the two neighborhoods, and testing for their intersection.

First, the algorithm will precompute some pieces of data of the two neighborhoods, and store them into the arrays *nba* and *nbb* for later use. Similarly to the splitting algorithm, it is very helpful to subdivide "wide" sectors whose bounding vectors make an angle larger than 180 degrees. These tasks are implemented by the Program 15.8

The main loop of the Program 15.7 compares two preprocessed sectors by means of the procedure `sectortest` described in the next subsection. If the two sectors are deemed to intersect, the classifications (`IN`, `ON`, `OUT`) of the bounding vertices of each sector with respect to the other sector are computed and stored into the array *sectors* for later processing.

Sector intersection test

The sector intersection test is illustrated in Figure 15.7. A vector *int* along the intersection line can be calculated as the cross product of the plane normal vectors *n1* and *n2*:

$$int = n1 \times n2 \tag{15.4}$$

If the cross product vanishes, the sectors are coplanar and considered to intersect if they overlap. Otherwise, an inclusion test for *int* and the sectors is performed; if *int* occurs within both sectors, they intersect.

As shown in Figure 15.8, the inclusion is determined by examining the following cross products of *ref1*, *ref2*, and *int*:

$$\begin{aligned} ref &= ref1 \times ref2 \\ test1 &= ref1 \times int \\ test2 &= int \times ref2 \end{aligned} \tag{15.5}$$

If all three vectors are collinear, *int* is within the sector bounded by *ref1* and *ref2*.


```
void      setopgetneighborhood(va, vb)
Vertex    *va, *vb;
{
     HalfEdge   *ha, *hb;
     double     d1, d2, d3, d4;
     int        na, nb, i, j;

     nnba = nbrpreproc(va, nba);
     nnbb = nbrpreproc(vb, nbb);
     nsectors = 0;
     for(i=0; i<nnba; i++)
     {
          for(j=0; j<nnbb; j++)
          {
               if(sectortest(i, j))
               {
                    sectors[nsectors].secta = i;
                    sectors[nsectors].sectb = j;
                    d1 = dot(nbb[j].he->wloop->lface->feq,
                             nba[i].ref1);
                    d2 = dot(nbb[j].he->wloop->lface->feq,
                             nba[i].ref2);
                    d3 = dot(nba[i].he->wloop->lface->feq,
                             nbb[j].ref1);
                    d4 = dot(nba[i].he->wloop->lface->feq,
                             nbb[j].ref2);
                    sectors[nsectors].s1a = comp(d1, 0.0, EPS);
                    sectors[nsectors].s2a = comp(d2, 0.0, EPS);
                    sectors[nsectors].s1b = comp(d3, 0.0, EPS);
                    sectors[nsectors].s2b = comp(d4, 0.0, EPS);
                    sectors[nsectors++].intersect = 1;
               }
          }
     }
}
```

Program 15.7 Search for intersecting sectors.

```
int             nbrpreproc(v, n)
Vertex          *v;
struct nb       n[];
{
    vector      bisec;
    HalfEdge    *he;
    int         i;

    i = 0;
    he = v->vedge;
    do
    {
        n[i].he = he;
        vecminus(n[i].ref1, he->prv->vtx->vcoord,
                 he->vtx->vcoord);
        vecminus(n[i].ref2, he->nxt->vtx->vcoord,
                 he->vtx->vcoord);
        cross(n[i].ref12, n[i].ref1, n[i].ref2);
        if(vecnull(n[i].ref12, EPS) ||
          (dot(n[i].ref12, he->wloop->lface->feq) > 0.0))
        {
            if(vecnull(n[i].ref12, EPS))
                inside(he, bisec);
            else
            {
                vecplus(bisec, n[i].ref1, n[i].ref2);
                bisec[0] = -bisec[0];
                bisec[1] = -bisec[1];
                bisec[2] = -bisec[2];
            }
            n[i+1].he = he;
            veccopy(n[i+1].ref2, n[i].ref2);
            veccopy(n[i+1].ref1, bisec);
            veccopy(n[i].ref2, bisec);
            cross(n[i].ref12, n[i].ref1, n[i].ref2);
            cross(n[i+1].ref12, n[i+1].ref1, n[i+1].ref2);
            i++;
        }
        i++;
    }
    while((he = mate(he)->nxt) != v->vedge);
    return(i);
}
```

Program 15.8 Preprocessing of sectors.

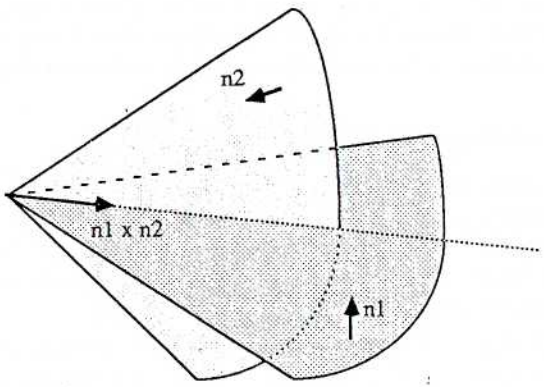

Figure 15.7 Sector intersection test.

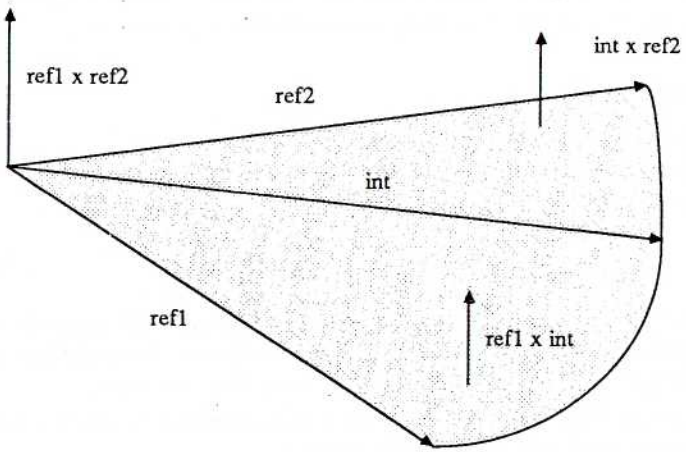

Figure 15.8 Within-sector test.

```
int  sectortest(i, j)
int  i, j;
{
     vector    intrs;
     HalfEdge  *h1, *h2;
     int       c1, c2;

     h1 = nba[i].he;
     h2 = nbb[j].he;
     cross(intrs, h1->wloop->lface->feq, h2->wloop->lface->feq);
     if(vecnull(intrs, EPS)) return(sectoroverlap(h1, h2));
     c1 = sctrwithin(intrs, nba[i].ref1, nba[i].ref2, nba[i].ref12);
     c2 = sctrwithin(intrs, nbb[j].ref1, nbb[j].ref2, nbb[j].ref12);
     if(c1 && c2) return(1);
     else
     {
          intrs[0] = -intrs[0];
          intrs[1] = -intrs[1];
          intrs[2] = -intrs[2];
          c1 = sctrwithin(intrs, nba[i].ref1,
                          nba[i].ref2, nba[i].ref12);
          c2 = sctrwithin(intrs, nbb[j].ref1,
                          nbb[j].ref2, nbb[j].ref12);
          if(c1 && c2) return(1);
     }
     return(0);
}

int      sctrwithin(dir, ref1, ref2, ref12)
vector   dir, ref1, ref2, ref12;
{
     vector  c1, c2;
     int     t1, t2;

     cross(c1, dir, ref1);
     if(vecnull(c1, EPS)) return(dot(ref1, dir) > 0.0);
     cross(c2, ref2, dir);
     if(vecnull(c2, EPS)) return(dot(ref2, dir) > 0.0);
     t1 = comp(dot(c1, ref12), 0.0, EPS);
     t2 = comp(dot(c2, ref12), 0.0, EPS);
     return(t1<0 && t2<0);
}
```

Program 15.9 Sector intersection test.

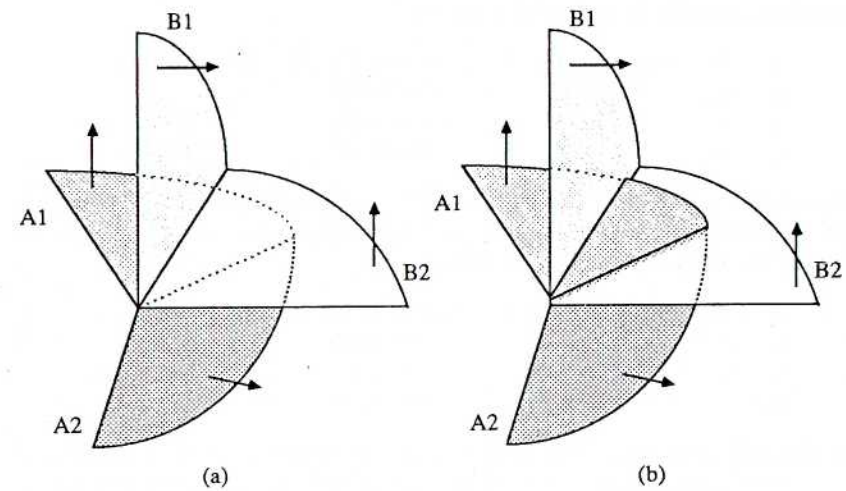

Figure 15.9 Reclassification on "on"-sectors.

The sector intersection and inclusion tests are implemented in Program 15.9. Most of the code is used to deal with various "special" cases (such as the case that one of the cross products of Equations 15.5 vanishes). The procedure **sectoroverlap** is assumed to check whether two coplanar sectors overlap. This is not elaborated on here.

Reclassification

The set of potentially intersecting sector pairs found by the sector intersection test will in general include various "special" cases, such as coplanar overlapping sectors and sectors that just touch each other along a bounding edge. Similarly to the classifier for the splitting problem, the next phase of the vertex-vertex classifier will reclassify such "on"-cases as "in" or "out" according to the rules of Equations 15.3.

Reclassification of "on"-sectors In strict analogy to the splitting algorithm, the reclassification can proceed by first reclassifying all "on"-sectors and their neighbors.

To come up with suitable reclassification rules, it is useful first to consider an example. Consider the arrangement of four sectors a_1, a_2 of solid A and b_1, b_2 of solid B depicted in Figure 15.9(a) that includes two overlapping coplanar sectors (a_1, b_2); arrows indicate the face normals.

After the sector intersection test, we have essentially the following information available in the array *sectors*:

A	B	Side code for A	Side code for B	intersect bit
a_1	b_1	IN-OUT	OUT-ON	1
a_1	b_2	ON-ON	ON-ON	1
a_2	b_2	ON-IN	IN-OUT	1

If we intend to calculate the set union, the rules of Equations 15.3 tell us that a_1 should be reclassified as lying outside B, and b_2 as lying inside A. That is, the table would read as follows:

A	B	Side code for A	Side code for B	intersect bit
a_1	b_1	IN-OUT	OUT-ON	1
a_1	b_2	OUT-OUT	IN-IN	0
a_2	b_2	ON-IN	IN-OUT	1

But this will allow us also to reclassify the information on the neighboring sectors—for instance, as indicated in Figure 15.9, a_2 should be treated as if it intersects b_2 and its side code should read "OUT-IN". This leads us to the following result:

A	B	Side code for A	Side code for B	intersect bit
a_1	b_1	IN-OUT	OUT-IN	1
a_1	b_2	OUT-OUT	IN-IN	0
a_2	b_2	OUT-IN	IN-OUT	1

Now, no "on"-cases remain, and we are done. Observe that as soon as we note that a sector is completely in one side of another, we can reset the intersection indicator.

The sector reclassification rules outlined in the sample case are implemented in Program 15.10. The algorithm simply loops through the array and locates coplanar sector pairs. If one is found, the new classifications for the sector edges are computed according to Equations 15.3. In the second "for"-loop, the neighbor sectors that can be reclassified are located and properly marked. Finally, the coplanar sectors themselves are reclassified.

Reclassification of "on"-edges After all coplanar sectors and their neighbors are classified, single "on"-edges may still remain. As indicated in Figure 15.10, there are two cases to be dealt with:

1. The "on"-edge lies within a sector of the other solid (Figure 15.10(a)).
2. The "on"-edge lies on the common edge of two sectors of the other solid (Figure 15.10(b)).


```
void  sreclsectors(op)
int   op;
{
      HalfEdge      *ha, *hb;
      int           i, j, nonopposite, newsa, newsb;
      int           secta, prevsecta, nextsecta;
      int           sectb, prevsectb, nextsectb;

      for(i=0; i<nsectors; i++)
      {
            if(sectors[i].s1a == ON && sectors[i].s2a == ON &&
               sectors[i].s1b == ON && sectors[i].s2b == ON)
            {
                  secta = sectors[i].secta;
                  sectb = sectors[i].sectb;
                  prevsecta = (secta == 0) ? nnba-1 : secta-1;
                  prevsectb = (sectb == 0) ? nnbb-1 : sectb-1;
                  nextsecta = (secta == nnba-1) ? 0 : secta+1;
                  nextsectb = (sectb == nnbb-1) ? 0 : sectb+1;

                  ha = nba[secta].he;
                  hb = nbb[sectb].he;
                  nonopposite = vecequal(ha->wloop->lface->feq,
                                    hb->wloop->lface->feq, EPS);
                  if(nonopposite)
                  {
                        newsa = (op == UNION) ? OUT : IN;
                        newsb = (op == UNION) ? IN : OUT;
                  }
                  else
                  {
                        newsa = (op == UNION) ? IN : OUT;
                        newsb = (op == UNION) ? IN : OUT;
                  }
                  for(j=0; j<nsectors; j++)
                  {
                        if((sectors[j].secta == prevsecta) &&
                           (sectors[j].sectb == sectb))
                              if(sectors[j].s1a != ON)
                                    sectors[j].s2a = newsa;
```

Program 15.10 Reclassification of "on"-sectors.

```
                if((sectors[j].secta == nextsecta) &&
                   (sectors[j].sectb == sectb))
                       if(sectors[j].s2a != ON)
                              sectors[j].s1a = newsa;
                if((sectors[j].secta == secta) &&
                   (sectors[j].sectb == prevsectb))
                       if(sectors[j].s1b != ON)
                              sectors[j].s2b = newsb;
                if((sectors[j].secta == secta) &&
                   (sectors[j].sectb == nextsectb))
                       if(sectors[j].s2b != ON)
                              sectors[j].s1b = newsb;
                if(sectors[j].s1a == sectors[j].s2a &&
                   (sectors[j].s1a == IN ||
                    sectors[j].s1a == OUT))
                      sectors[j].intersect = 0;
                if(sectors[j].s1b == sectors[j].s2b &&
                   (sectors[j].s1b == IN ||
                    sectors[j].s1b == OUT))
                      sectors[j].intersect = 0;
            }
            sectors[i].s1a = sectors[i].s2a = newsa;
            sectors[i].s1b = sectors[i].s2b = newsb;
            sectors[i].intersect = 0;
        }
    }
}
```

Program 15.10 Reclassification of "on"-sectors (cont.).

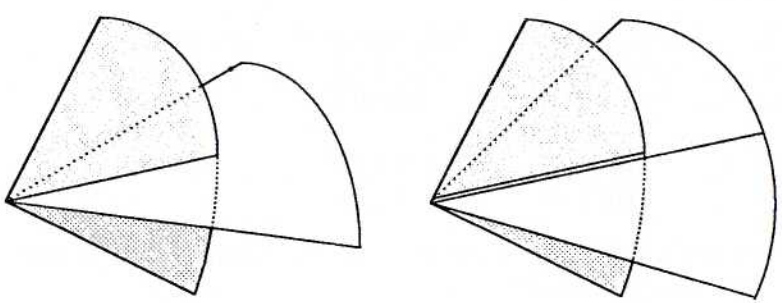

Figure 15.10 Reclassification of "on"-edges.

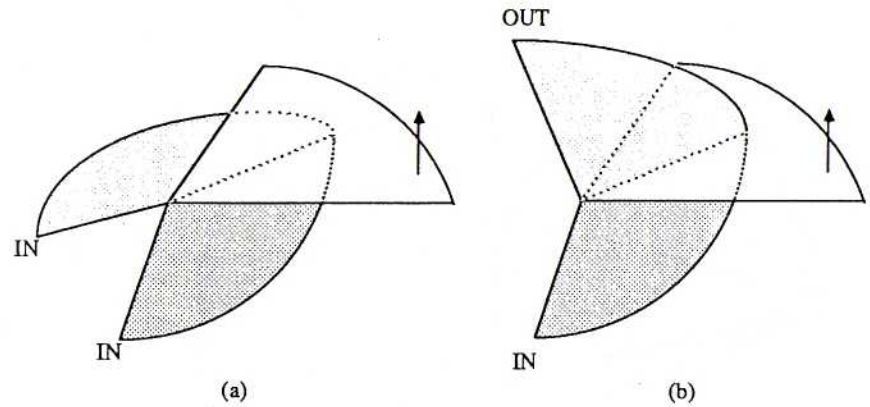

Figure 15.11 Simple "on"-edges.

The first case can be simply solved by looking at the side codes of the sectors; see Figure 15.11. In the case (a), the sectors are deemed to intersect, and in case (b) no intersection is recorded.

The second case is somewhat more complicated, because we need to look at four sectors. In this case, the existence of an intersection can be determined by sorting the four sectors meeting at the common line angularly around the line; if the sectors appear in a "mixed order," an intersection exists. In Figure 15.12, two arrangements with sectors sharing an edge are depicted on the left, and on the right the same arrangements are shown as viewed along the coincident edge. Clearly, in the first case the sectors intersect, whereas in the second case they do not.

Space does not permit the inclusion of the procedure **srecledges** expected to implement these reclassification rules.

Insertion of null edges

After reclassification, the final result of vertex-vertex classification is available in the array *sectors* as a sequence of intersecting sector pairs. The final part of the classifier inserts the appropriate null edges into the neighborhood so that the intersection polygons can be generated.

At this point, let us again look at the final result of the sample case of Figure 15.9:

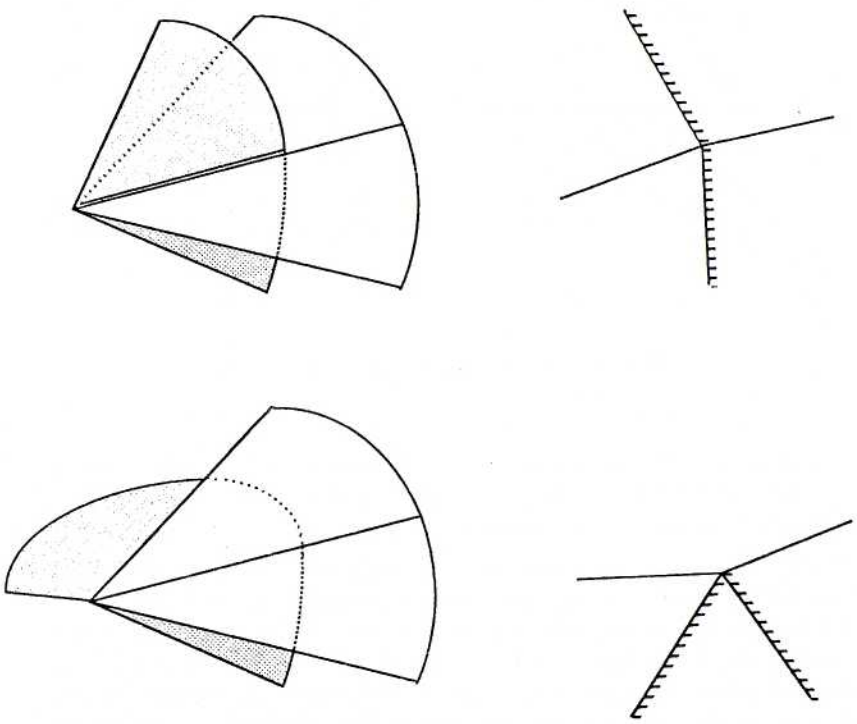

Figure 15.12 Double "on"-edges.

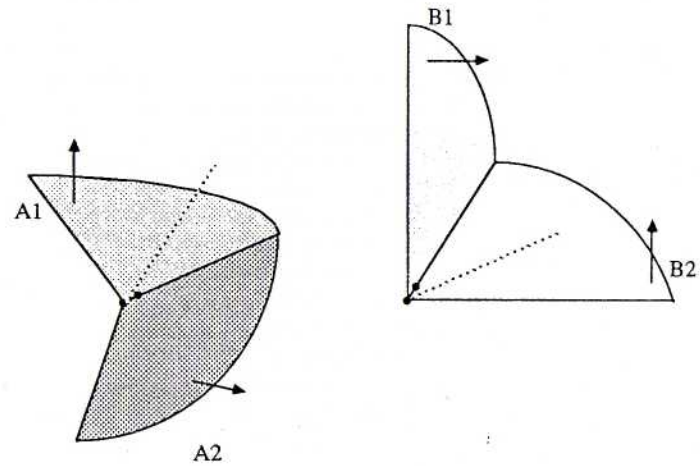

Figure 15.13 Insertion of null edges.

A	B	Side code for A	Side code for B	intersect bit
a_1	b_1	IN-OUT	OUT-IN	1
a_1	b_2	OUT-OUT	IN-IN	0
a_2	b_2	OUT-IN	IN-OUT	1

The middle row can be readily ignored, because its intersection indicator is set to zero. This leaves us with a pair of intersections, and it is easy to determine that the arrangement of Figure 15.13 should result.

The only other case that must be dealt with is the case when a single sector has several intersections. If so, one or the other of the following arrangements will be located:

A	B	Side code for A	Side code for B	intersect bit
a_1	b_1	IN-OUT	OUT-IN	1
a_1	b_2	OUT-IN	IN-OUT	1
a_1	b_1	IN-OUT	OUT-IN	1
a_2	b_1	OUT-IN	IN-OUT	1

In these cases, the correct action is to insert a "strut" null edge into the sector a_1 or b_1, respectively.

The insertion of null edges is implemented in Program 15.11. In addition to performing the case analysis discussed, the procedure rearranges

```
void    sinsertnulledges()
{
    HalfEdge    *ha1, *ha2, *hb1, *hb2;
    int         i = 0;

    while(1)
    {
        while(sectors[i].intersect == 0)
            if(++i == nsectors) return;
        if(sectors[i].s1a == OUT)
                ha1 = nba[sectors[i].secta].he;
        else    ha2 = nba[sectors[i].secta].he;
        if(sectors[i].s1b == IN)
                hb1 = nbb[sectors[i++].sectb].he;
        else    hb2 = nbb[sectors[i++].sectb].he;

        while(sectors[i].intersect == 0)
            if(++i == nsectors) return;
        if(sectors[i].s1a == OUT)
                ha1 = nba[sectors[i].secta].he;
        else    ha2 = nba[sectors[i].secta].he;
        if(sectors[i].s1b == IN)
                hb1 = nbb[sectors[i++].sectb].he;
        else    hb2 = nbb[sectors[i++].sectb].he;

        if(ha1 == ha2)
        {
            separ2(ha1, 0);
            separ1(hb1, hb2, 1);
        }
        else if(hb1 == hb2)
        {
            separ2(hb1, 1);
            separ1(ha2, ha1, 0);
        }
        else
        {
            separ1(ha2, ha1, 0);
            separ1(hb1, hb2, 1);
        }
        if(i == nsectors) return;
    }
}
```

Program 15.11 Insertion of null edges.


```
void            separ1(from, to, type)
HalfEdge        *from, *to;
int             type;
{
        lmev(from, to, ++maxv,
             from->vtx->vcoord[0],
             from->vtx->vcoord[1],
             from->vtx->vcoord[2]);
        if(type == 0) sonea[nedga++] = from->prv->edg;
        else          soneb[nedgb++] = from->prv->edg;
}

void            separ2(he, type)
HalfEdge        *he;
int             type;
{
        HalfEdge        *tmp;

        separ1(he, he, type);
}
```

Program 15.12 Auxiliaries for the insertion of null edges.

the sectors according to the side codes in order to to give a consistent orientation to the null edges.

The actual insertion of null edges is accomplished by the procedures **separ1** and **separ2** shown in Program 15.12. These procedures also manage the arrays storing the null edges.

15.7 JOINING OF NULL EDGES

Let us now consider the procedure **setopconnect** which is expected to combine the null edges generated by the previous steps in order to create the intersection polygons.

This task is obviously similar to the joining problem for the splitting algorithm, with the exception that the intersection polygons are in general not planar figures. Therefore, we cannot expect that the very same algorithm would work for set operations.

Nevertheless, there is a further property of the set operations context that we can exploit. Recall that null edges are always generated in pairs, one into solid A and one into solid B. Of course, the intersection polygons

should be constructed in the same fashion in each solid—that is, in the same order through the corresponding null edges.

This property can be used to come up with an algorithm for combining the null edges for set operations. The procedure **scanjoin** of Program 15.13 is a modification of the procedure **canjoin** (Program 14.9) of the splitting algorithm that considers two corresponding null edges at one time, and considers them "joinable" only if both can be joined to a loose end. Current loose ends are maintained in two arrays, one for solid A and one for solid B. If the edges cannot be joined, they are added to the set of loose ends; clearly, the updating of the arrays is implemented in a fashion that maintains the correspondence.

Based on **scanjoin**, the joining algorithm for the splitting algorithm can almost mechanically be modified for the set operations. The resulting procedure **setopconnect** is given in Program 15.14. After sorting the two arrays of null edges, the algorithm processes the pairs of corresponding null edges one at a time (by means of procedure **sgetnextnulledge**), and performs the correct actions similarly as the splitting algorithm. The procedures **isloosea**, **islooseb**, **cuta**, and **cutb** are simple modifications of the corresponding procedures **isloose** and **cut** of the spliting algorithm, and we will not elaborate on them.

15.8 GENERATION OF THE RESULT

The final phase of the set operations algorithm is expected to actually apply the Equations 15.1 so as to create the final results.

The procedures already developed for the splitting problem are readily useful for the current task as well. The algorithm of Program 15.15 first divides each null face into two intersection faces by means of a **lmfkrh**.

On the basis of the consistent orientation of null edges, the new faces created by **lkfmrh** (**sonfa[nfaca+i]**, **sonfb[nfacb+i]**) are known to belong to the "in"-components ($AinB$, $BinA$) of the boundary classification of A and B, while the remaining faces (**sonfa[i]**, **sonfb[i]**) belong to the "out" components.

The algorithm exploits this fact by using the procedure **movefac** of Program 14.12 to move the desired component into the resulting solid *res*. Thereafter, the procedure **cleanup** reconstructs the edge and vertex lists of the result. As the final step, the intersection faces are combined by means of a **lkfmrh** and their edges and vertices merged by means of **loopglue**.

The set difference poses a slight difficulty because the orientation of the "in" component must first be reversed (see Equations 15.1). This is implemented by creating a temporary solid *ainb*, and reversing its orientation


```
static HalfEdge *endsa[30];
static HalfEdge *endsb[30];
static int      nenda;
static int      nendb;

int             scanjoin(hea, heb, reta, retb)
HalfEdge        *hea, *heb, **reta, **retb;
{
        int             i, j;

        for(i = 0; i<nenda; i++)
        {
                if(neighbor(hea, endsa[i]) &&
                   neighbor(heb, endsb[i]))
                {
                        *reta = endsa[i];
                        *retb = endsb[i];
                        for(j=i+1; j<nenda; j++)
                        {
                                endsa[j-1] = endsa[j];
                                endsb[j-1] = endsb[j];
                        }
                        nenda--;
                        nendb--;
                        return(1);
                }
        }
        endsa[nenda++] = hea;
        endsb[nendb++] = heb;
        *reta = NIL;
        *retb = NIL;
        return(0);
}
```

Program 15.13 Joinability test for set operations.

```
void    setopconnect()
{
        Edge            *nextedgea, *nextedgeb;
        HalfEdge        *h1a, *h2a, *h1b, *h2b;

        nfaca = nfacb = 0;
        ssortnulledges();
        while(sgetnextnulledge(&nextedgea, &nextedgeb))
        {
                if(scanjoin(nextedgea->he1,
                   nextedgeb->he2, &h1a, &h2b))
                {
                        join(h1a, nextedgea->he1);
                        if(!isloosea(mate(h1a)))
                                cuta(h1a);
                        join(h2b, nextedgeb->he2);
                        if(!islooseb(mate(h2b)))
                                cutb(h2b);
                }
                if(scanjoin(nextedgea->he2,
                   nextedgeb->he1, &h2a, &h1b))
                {
                        join(h2a, nextedgea->he2);
                        if(!isloosea(mate(h2a)))
                                cuta(h2a);
                        join(h1b, nextedgeb->he1);
                        if(!islooseb(mate(h1b)))
                                cutb(h1b);
                }
                if(h1a && h1b && h2a && h2b)
                {
                        cuta(nextedgea->he1);
                        cutb(nextedgeb->he1);
                }
        }
}
```

Program 15.14 Joining algorithm for set operations.


```
void    setopfinish(a, b, res, op)
Solid   *a, *b, **res;
int     op;
{
        int             i, j, inda, indb;

        inda = (op == INTERSECT) ? nfaca : 0;
        indb = (op == UNION)     ? 0 : nfacb;
        *res = (Solid *) new(SOLID, NIL);
        for(i=0; i<nfaca; i++)
        {
                sonfa[nfaca+i] = lmfkrh(sonfa[i]->floops->nextl,
                     ++maxf);
                sonfb[nfacb+i] = lmfkrh(sonfb[i]->floops->nextl,
                     ++maxf);
        }
        if(op == DIFFERENCE)
                revert(b);
        for(i=0; i<nfaca; i++)
        {
                movefac(sonfa[i+inda], *res);
                movefac(sonfb[i+indb], *res);
        }
        cleanup(*res);
        for(i=0; i<nfaca; i++)
        {
                lkfmrh(sonfa[i+inda], sonfb[i+indb]);
                loopglue(sonfa[i+inda]);
        }
}
```

Program 15.15 Generation of the result.

by means of the procedure **revert** left as an exercise to the reader. After this, **merge** will do the right thing.

Observe that the parts of A and B not required in the final result will remain in the original solids. This means that the algorithm could be extended to calculate both $A \cup B$ and $A \cap B$, or $A \setminus B$ and $B \setminus A$, simultaneously.

15.9 FINAL REMARKS

As suggested by Requicha and Voelcker [96], the techniques for classification and neighborhood analysis developed for boundary evaluation of CSG models are applicable to evaluation Boolean operations of BR models as well. Indeed, the algorithm described can be regarded as a particular way to implement the theoretical procedures Requicha and Voelcker outline in their article. However, while they concentrate on explicitly represented face and edge neighborhoods, our algorithm is based on implicitly represented vertex neighborhoods. Moreover, our algorithm is much more economical by exploiting the adjacency information usually available in "data-rich" boundary representations such as the winged-edge representation, and by avoiding point-in-polyhedron tests completely (except for the case that the surfaces of the solids no not intersect at all).

The algorithm does not extend directly to objects with curved faces or to nonmanifold objects. Curved faces may intersect each other in a way that cannot be reduced to vertex-vertex coincidences (consider spheres, for instance). Set operations of nonmanifolds would require the classification of several vertex neighborhoods from one object against several neighborhoods from the other, instead of the one-against-one case handled by this algorithm.

BIBLIOGRAPHIC NOTES

As noted in the introduction of this chapter, the literature of algorithms for Boolean set operations is meager and often quite inaccurate. Nevertheless, some traces can be given to the interested reader.

The algorithm described can be viewed as an implementation of the methods described in [73]; it is an offspring of earlier algorithms described in [117,79]. The idea of reclassification of "on"-entities appears in various levels of explicitness in several publications, such as [27,135].

As presented, the algorithm is slow. A practical Boolean set operations algorithm should definitely employ various methods for speeding up the computation by means of enclosing boxes and similar aids. A careful implementation of the

edge-face comparison of Section 15.4 can speed the whole algorithm up by a factor of 10 or so, but the quadratic basic behavior still remains.

However, as noted by Requicha and Voelcker, the worst-case analysis of the computational efficiency for set operations is not very informative as for the practical speed of the algorithm, and advanced geometric search techniques can be applied to it so as to exploit the typical geometric locality of the computation. An example of this is described in [79].

Figures 15.2 through 15.12 have been previously published in the article "Boolean operations of 2-manifolds through vertex neighborhood classification" by M. Mäntylä in *Transactions on Graphics*, Volume 5, Number 1. ©1986, Association for Computing Machinery, Inc. Reprinted by permission.

PROBLEMS

15.1. Program 15.1 does not deal properly with the case that one solid is completely within the other. Generalize the algorithm so that it will produce the correct results even in these cases.

Hint: You will need a procedure for testing the inclusion of a vertex within a solid.

15.2. Implement the procedure

```
void    maximize_faces(s)
Solid   *s;
```

that removes all inessential edges (i.e., edges that separate two coplanar faces, or occur within just a single face) of solid s.

Hint: Observe that you have to deal with special cases such as a face lying completely within another.

15.3. Reimplement the algorithm for edge-face comparison so that the recursion is eliminated.

Hint: Gather all intersections of the query edge into an auxiliary data structure. After all comparisons, sort the intersections along the edge, and process them in one sweep.

15.4. Implement the vertex-face classifier for set operations by modifying Programs 14.3–7 as suggested in Section 15.6.1. (With some modifications to the splitting algorithm, the *same* code could actually be used.)

15.5. Implement the procedure **sectoroverlap((h1, h2))** that tests whether two coplanar sectors **h1** and **h2** overlap.

15.6. Implement the procedure **revert(s)** that reverses the orientation of its argument solid s.

Hint: This task is best implemented through direct manipulation of the data structure. You will have to reverse the orientation of all loops and face equations.

15.7. Extend the algorithm to calculate both $A \cup B$ and $A \cap B$, or $A \setminus B$ and $B \setminus A$, simultaneously.

15.8. On the basis of the three-dimensional set operations algorithm presented, write a two-dimensional set operations algorithm that works on coplanar laminae.

Hint: Your algorithm can start by sweeping the laminae so as to create three-dimensional solids, and computing their Boolean combination. After that, the procedure **maximize_faces** of Problem 15.2 and the procedure **unsweep** of Problem 12.4 on page 216 can be used to generate the desired result.

15.9. Test your understanding: Would the set operations algorithm correctly compute $A \setminus B$ as the set intersection of A and reversed B?